PyQGIS - Python no QGIS

Disciplina: Programação aplicada à engenharia cartográfica

Maurício C. M. de Paulo - D.Sc.

18 de março de 2026

Arquitetura do QGIS

QGIS é estruturado em camadas (layers de software):

- **Core** → Modelo de dados geoespacial
- **GUI** → Interface gráfica (Qt)
- **Analysis** → Algoritmos e Processing
- **Providers** → OGR, GDAL, PostGIS, etc.

PyQGIS é o binding Python da API C++ do QGIS.

QgsApplication e Inicialização

Dentro do QGIS:

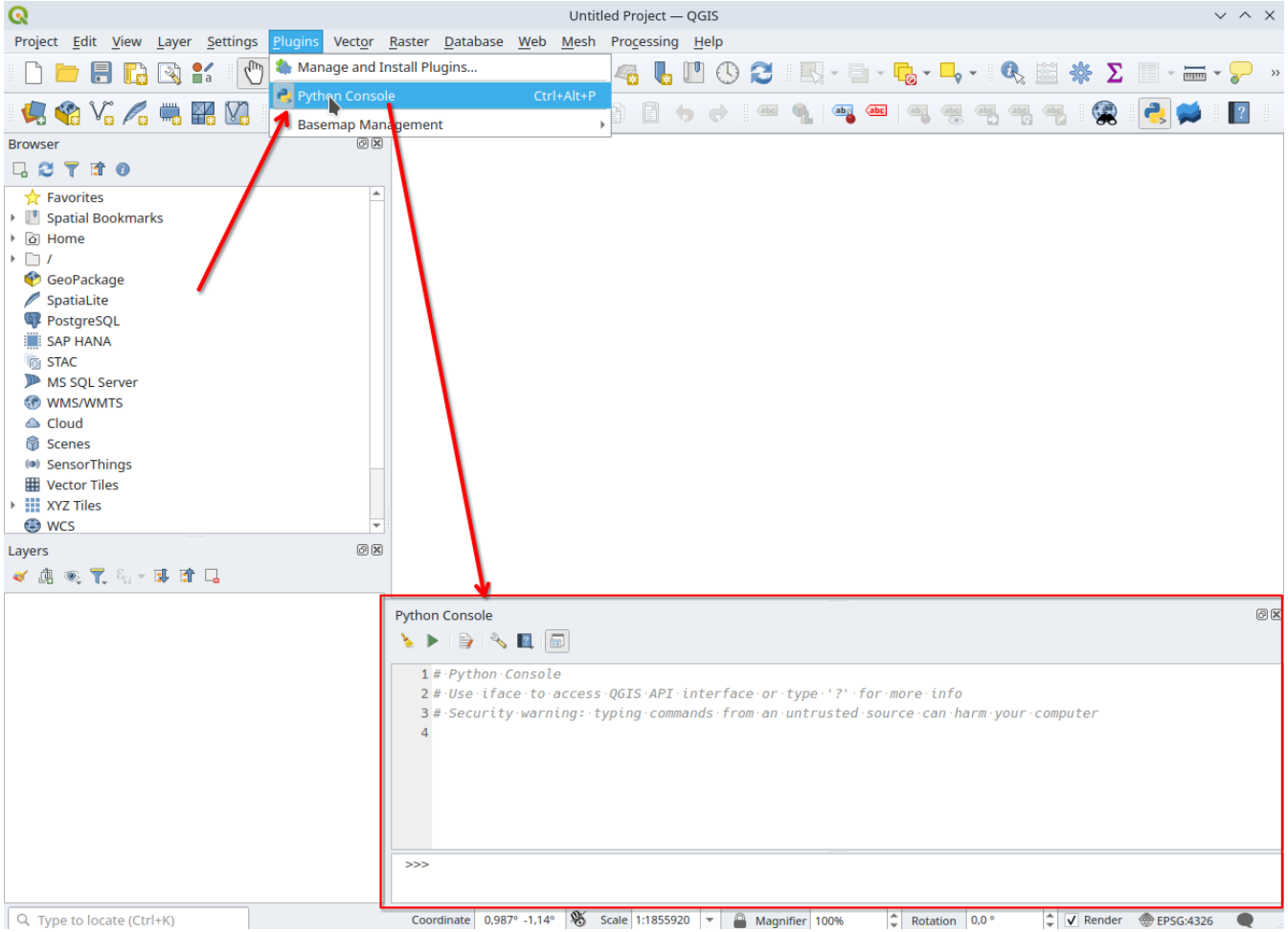
```
# Já inicializado automaticamente
layer = iface.activeLayer()
```

Script standalone:

```
from qgis.core import QgsApplication

QgsApplication.setPrefixPath("/usr", True)
app = QgsApplication([], False)
app.initQgis()

# código aqui

app.exitQgis()
```

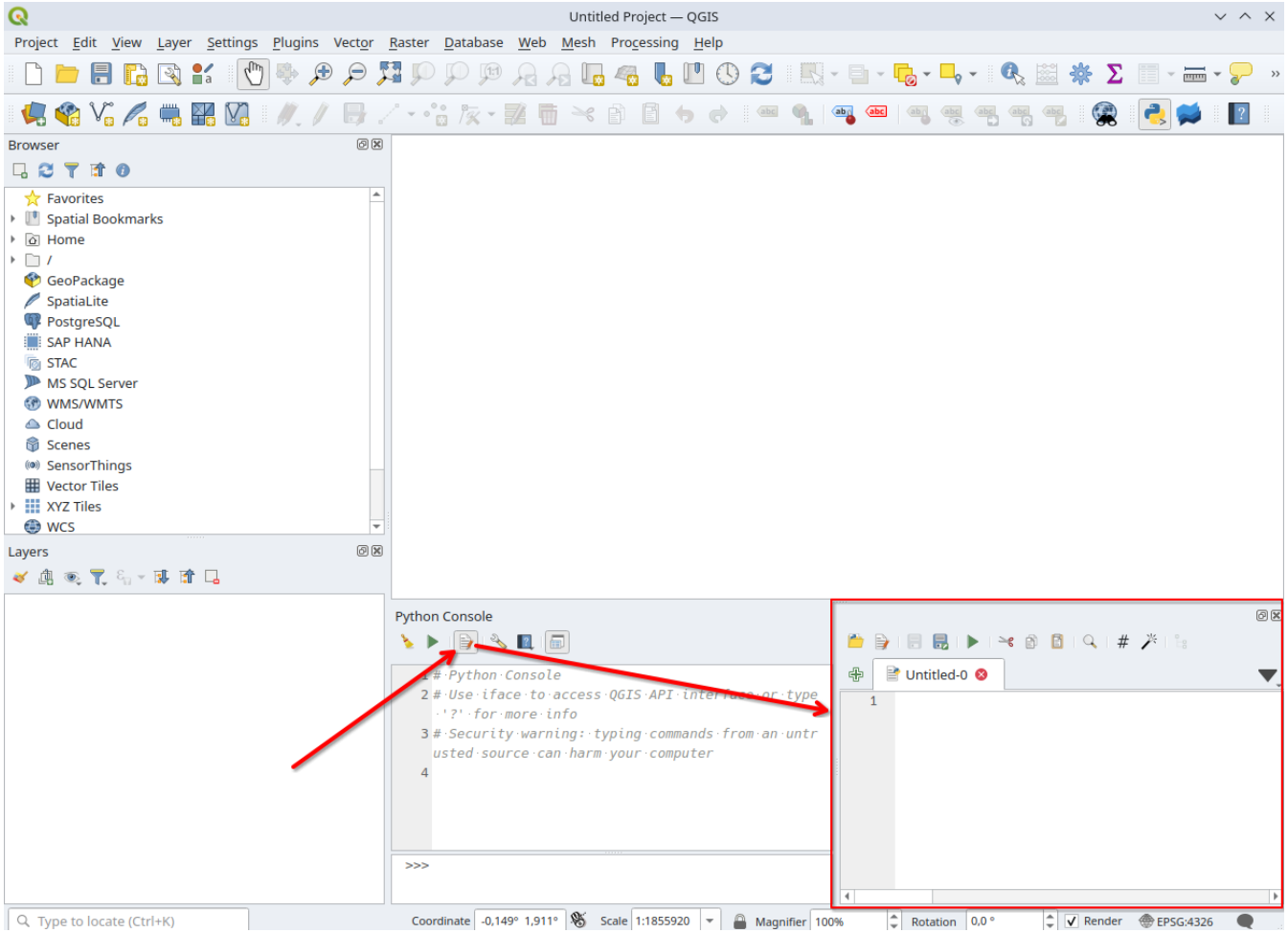
Scripts Python no QGIS

Terminal Python O terminal Python do QGIS permite executar comandos de forma interativa.



Scripts Python no QGIS

Editor de scripts Python O Editor de scripts Python do QGIS permite editar e executar scripts inteiros.



Manipulação da estrutura do Projeto no QGIS

QgsProject

- Representa o projeto aberto (.qgz)
- Gerencia camadas
- Singleton

Obter a instância (global) do projeto

```
from qgis.core import QgsProject
project = QgsProject.instance()
```

Permite:

- Adicionar camadas
- Remover camadas
- Acessar CRS do projeto

PyQGIS — QgsProject.instance()

QgsProject.instance() fornece acesso ao **projeto QGIS atualmente aberto**. É o ponto central para acessar **camadas, CRS, caminhos e estado do projeto**.

Gerenciamento de camadas

```
project.mapLayers()

project.mapLayersByName("rios")

project.addMapLayer(layer)

project.removeMapLayer(layer.id())
```

Informações do projeto

```
project.fileName()

project.homePath()

project.crs()

project.setCrs(crs)
```

Exemplo: listar camadas

```
for layer in project.mapLayers().values():
    print(layer.name())
```

Providers

Providers são responsáveis pelo acesso aos dados

- OGR → Shapefile, GeoPackage
- GDAL → Raster
- PostGIS → Banco espacial

Ao criar uma camada:

```
QgsVectorLayer(path, name, "ogr")
```

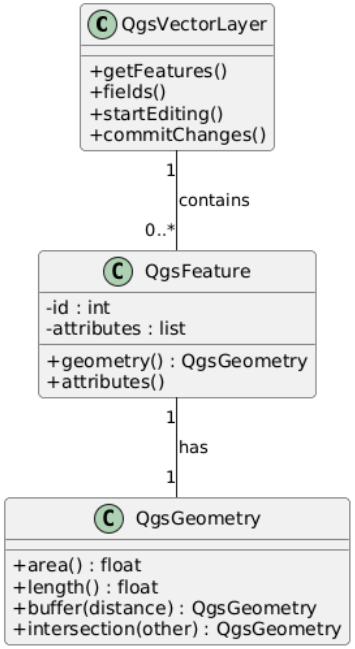
"ogr" é o provider.

Modelo de Dados Vetorial

Principais classes:

- **QgsVectorLayer**
- **QgsFeature**
- **QgsGeometry**
- **QgsField / QgsFields**

Diagrama de classes



Relação conceitual:

$$\text{Layer} \rightarrow \text{Features} \rightarrow \text{Geometry} + \text{Attributes}$$

QgsVectorLayer: diferentes fontes

A classe `QgsVectorLayer` permite acessar dados vetoriais de diversos formatos.

```
from qgis.core import QgsVectorLayer, QgsProject
```

Shapefile

```
layer = QgsVectorLayer(
    "/dados/rios.shp",
    "Nome da camada",
    "ogr"
)
```

GeoPackage

```
layer = QgsVectorLayer(
    "/dados/brasil.gpkg|"\
    "layername=estados",
    "Nome da camada",
    "ogr"
)
```

PostGIS

```
uri = "dbname='gis' host=localhost \
port=5432 user='postgres' \
"password='123' \
table=\"public\".\"municipios\" (geom)"

layer = QgsVectorLayer(uri,
    "Nome da camada",
    "postgres"
)
```

Adicionar ao projeto

```
QgsProject.instance().addMapLayer(layer)
```

QgsRasterLayer: diferentes fontes

A classe `QgsRasterLayer` permite acessar dados matriciais de diversas fontes.

```
from qgis.core import QgsRasterLayer, QgsProject
```

GeoTIFF

```
layer = QgsRasterLayer(
    "/dados/imagem.tif",
    "Nome da camada",
    "gdal"
)
```

COG (Cloud Optimized GeoTIFF)

```
url = "https://exemplo.com/imagem.tif"
layer = QgsRasterLayer(
    url,
    "Nome da camada",
    "gdal"
)
```

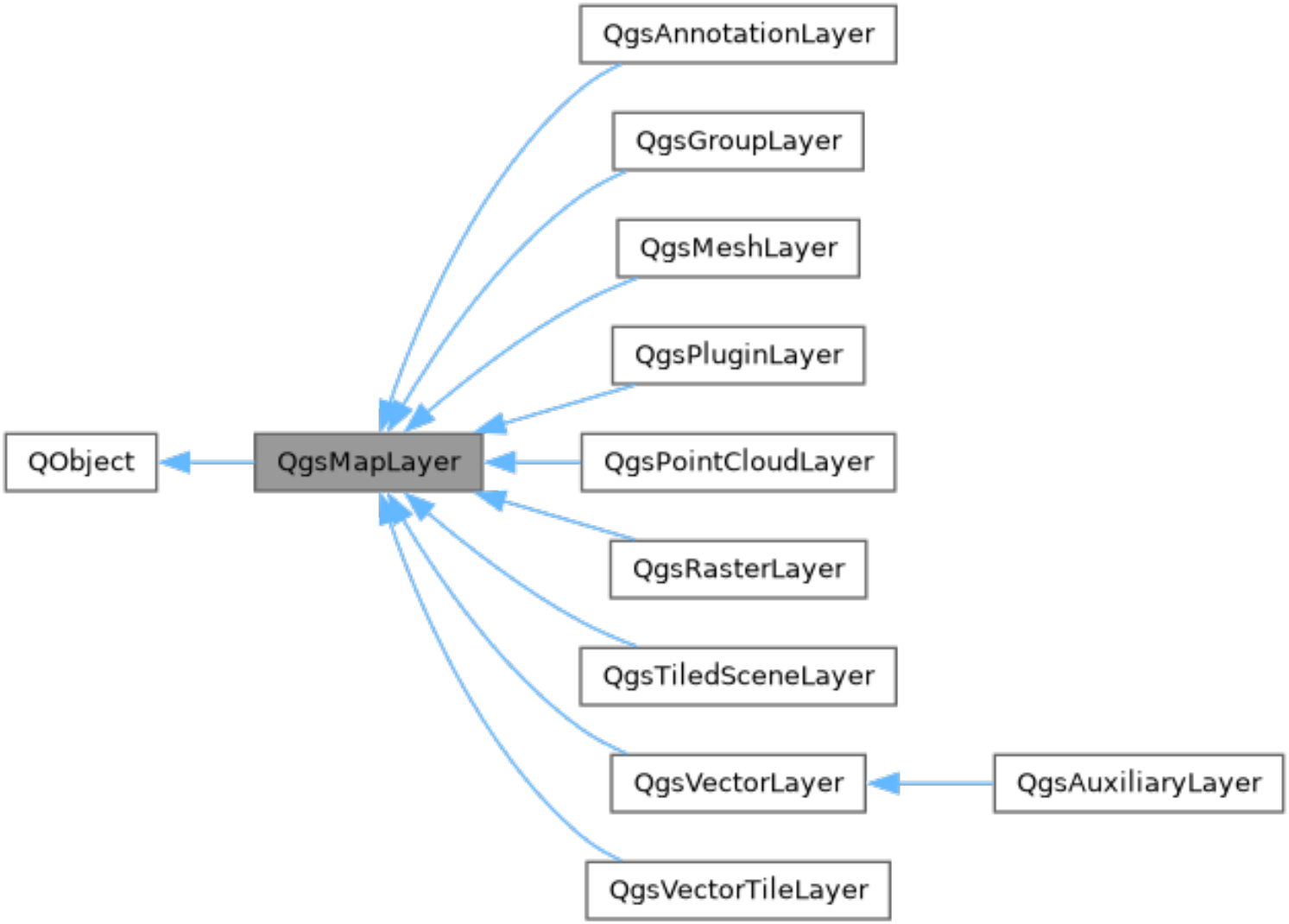
WMS

```
url = "crs=EPSG:4326&"\
      "format=image/png&"\
      "layers=ctm25&"\
      "styles&"\
      "url=https://bdgex.eb.mil.br/mapcac

layer = QgsRasterLayer(
    url,
    "Nome da camada",
    "wms"
)
```

Adicionar ao projeto

```
QgsProject.instance().addMapLayer(layer)
```

Tipos de camada no QGIS



https://api.qgis.org/api/classQgsMapLayer.html

Automação e Scripts Standalone

Objetivo:
Executar rotinas de geoprocessamento fora da interface gráfica do QGIS.

Casos típicos:

- Processamento em lote
- Integração com pipelines ETL (extract, transform, load)
- Execução em servidor
- Agendamento (cron / task scheduler)

Comparação: Standalone vs Plugin

	Plugin	Standalone
Interface gráfica	Sim	Não
Execução agendada	Limitado	Sim
Escalabilidade	Média	Alta
Uso em servidor	Difícil	Ideal

Arquitetura: GUI vs Standalone

Dentro do QGIS:

- iface disponível
- Projeto já carregado
- Ambiente inicializado

Standalone:

- Inicializar QApplication
- Inicializar QgsApplication
- Configurar prefixPath

Estrutura Mínima de Script Standalone

```python
from qgis.core import *
from qgis.PyQt.QtWidgets import QApplication
import sys

QgsApplication.setPrefixPath(
    "/usr", True
)

app = QApplication(sys.argv)
qgs = QgsApplication([], False)
qgs.initQgis()

# --- código aqui ---

qgs.exitQgis()
```

Componentes Críticos

QgsApplication:

- Inicializa provedores
- Carrega drivers GDAL/OGR
- Gerencia ambiente QGIS

setPrefixPath():

- Define onde o QGIS está instalado
- Essencial para localizar recursos internos

Executando Processing Standalone

```
import processing
from processing.core.Processing import Processing

Processing.initialize()

result = processing.run("native:buffer", {
    "INPUT": layer,
    "DISTANCE": 100,
    "OUTPUT": "/dados/buffer.shp"
})
```

Observação: Processing precisa ser explicitamente inicializado.

Automação em Lote

Padrão típico:

- Listar arquivos com glob/os
- Iterar camadas
- Executar algoritmo
- Salvar saída

Aplicações:

- Processamento municipal
- Geração de buffers por estado
- Reprojeção em massa

Integração com CLI

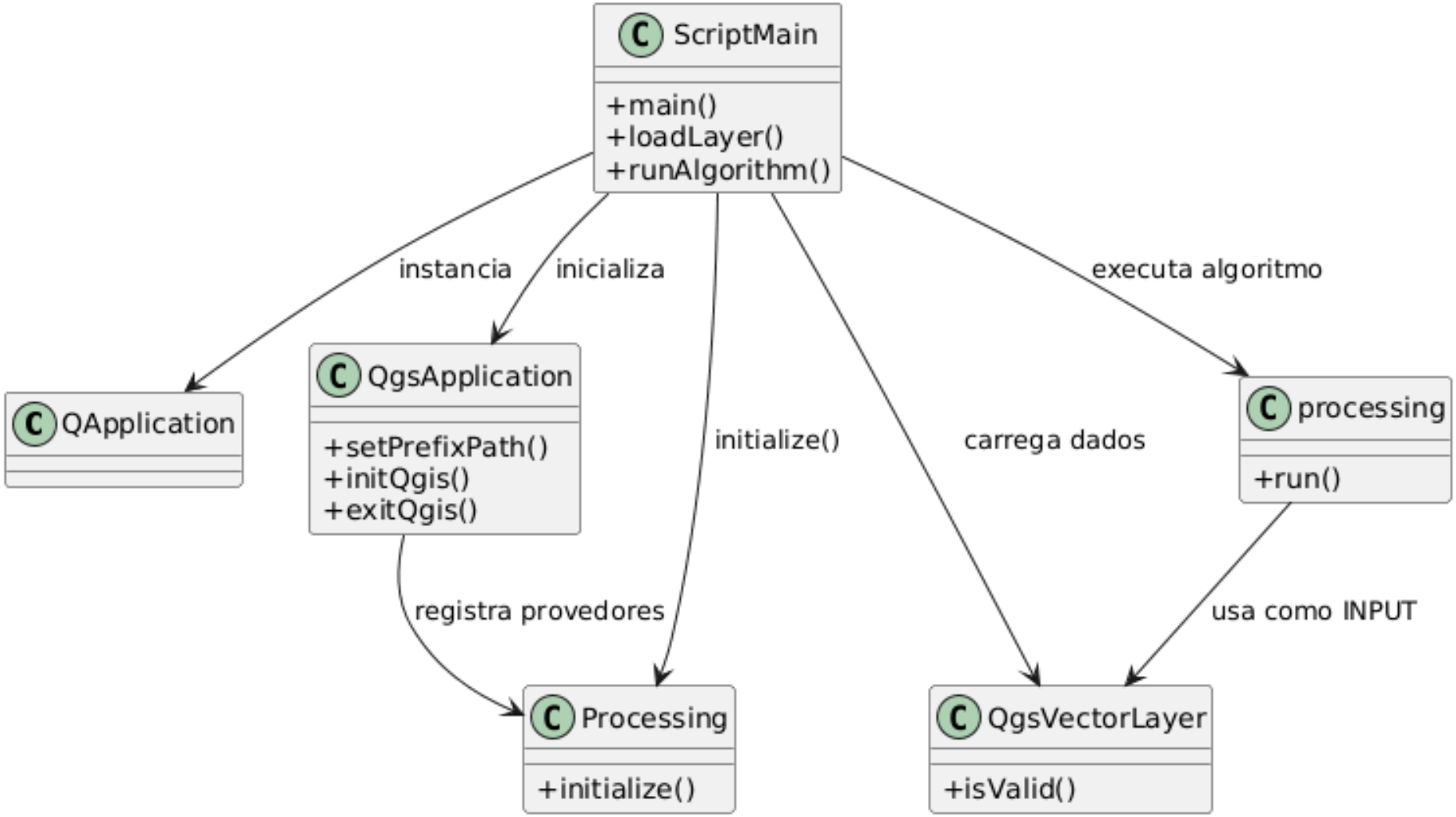
Script pode receber argumentos:

- argparse
- variáveis de ambiente
- arquivos de configuração (YAML/JSON)

Permite:

- Execução parametrizada
- Integração com Airflow
- Pipelines CI/CD

Boas Práticas

- Encapsular lógica em funções
- Evitar código global
- Validar camadas antes de processar
- Usar logs ao invés de print
- Garantir qgs.exitQgis()

Arquitetura Standalone — PyQGIS